GridMM_ff 阶段性实验总览

这份文档用于总结到目前为止在 `GridMM_ff` 上已经做过的主要改动、实验方向、结论，以及当前更值得继续推进的配置。

整体目标一直比较明确：

- 不训练
- 只做离散环境
- 优先做 test-time / inference-time 小改动
- 尽量少改代码
- 重点看 `SR / SPL / nDTW / lengths / steps`

1. 已经完成的代码改动

1.1 Dynamic Memory 机制

已经在离散任务主线里实现了一个最小的 dynamic memory 版本，支持：

- `OFF`
- `UPDATE_ONLY`
- `DECAY_ONLY`
- `FULL`

核心思路：

- 写入侧：给已有 memory slot 加 heuristic update gate
- 读出侧：给旧 memory 加 soft forgetting / decay
- 完全不引入新可学习参数
- 默认关闭，方便 ablation

主要落点：

- `map_nav_src/r2r/env.py`
- `map_nav_src/reverie/env.py`
- `map_nav_src/rxr/env.py`

- `map_nav_src/models/vilmodel.py`
- `map_nav_src/models/vlnbert_init.py`
- `map_nav_src/r2r/parser.py`
- `map_nav_src/reverie/parser.py`
- `map_nav_src/rxr/parser.py`

脚本层也已经支持：

- `DYNAMIC_MEMORY_MODE=off|update_only|decay_only|full`

1.2 Distributed / NCCL clean shutdown 修复

为了解决评测结束后终端不退出、 `destroy_process_group()` warning 的问题，已经做过一次统一收尾修复。

主要内容：

- 主入口加 `try/finally`
- `finally` 里统一调用 `cleanup_distributed()`
- 单卡时不再无意义初始化 distributed
- cleanup 不再依赖容易卡住的 barrier

主要落点：

- `map_nav_src/main_nav.py`
- `map_nav_src/main_nav_obj.py`
- `map_nav_src/main_rxr.py`
- `map_nav_src/main.py`
- `map_nav_src/utils/distributed.py`

这部分已经验证过：

- 单卡 eval 可以正常退出
- 不再出现 `destroy_process_group() was not called...` 警告

1.3 R2R: dual STOP 机制

在 `R2R` 上做过一版最小 dual STOP：

- 语义条件：`STOP` 分数足够高
- 几何 / 拓扑条件：步数、revisit、margin 等
- 只在 test-time 生效
- 同时对历史 `node_stop_scores` 做一致性 gate

主要落点：

- `map_nav_src/r2r/agent.py`
- `map_nav_src/r2r/parser.py`
- `map_nav_src/scripts/run_r2r.sh`

实验结论：

- 这条线整体上是负收益
- 典型现象：
  - `oracle_sr` 高
  - 但 `sr` / `spl` 掉
  - `lengths` / `steps` 变长
- 说明 agent 更常到达正确区域，但没有及时停下，反而继续走

当前结论：

- `dual stop` 不建议继续作为主优化线

1.4 R2R: instruction 线尝试

instruction 方向一共做过三类尝试。

1.4.1 instruction augmentation

做法：

- 不改模型结构
- 只在编码前给原 instruction 追加关键词摘要

结果：

- 明显负收益
- 说明直接修改文本输入会破坏原本 encoder 已适配的输入分布

结论：

- 不继续

1.4.2 global direction rerank

做法：

- 从整条 instruction 中提取 `left / right / straight`
- 在 early uncertain steps 对 non-stop action 做极弱 rerank

结果：

- 保守配置下几乎不改动作
- 激进 sanity check 下可以改动作，但指标没有变好

结论：

- 证明了“这条最小 rerank 机制能改动作”
- 但不能证明“改得更对”
- 不适合作为主线继续深挖

1.4.3 local first-clause rerank

做法：

- 只看第一子句
- 只认显式动作短语，如：
  - `turn left`
  - `turn right`
  - `go straight`

结果：

- 覆盖率大幅下降
- rerank 几乎不再触发
- 效果基本退化成 baseline

结论：

- 太保守
- 也不继续

这条 instruction 线已经整理成单独文档：

- `codex/instruction_line_summary.md`

1.5 R2R: anti-loop / immediate backtrack suppression

后来切换到一个更结构化的动作层约束：

- immediate backtrack suppression

核心不是惩罚 target viewpoint 本身，而是：

- 对“走向某个 target 时的第一跳 next-hop”做惩罚

当前实现里支持两层：

1. `immediate backtrack penalty`
   - 如果 `next_hop == prev_vpid`
   - 给较强惩罚
2. `revisit penalty`
   - 如果 `next_hop` 已访问次数超过阈值
   - 给较弱惩罚

但从实际统计看：

- 真正起作用的是 `immediate backtrack penalty`
- `revisit_penalty_count` 基本为 `0`

所以这条线后来已经收缩成：

- `immediate backtrack suppression`

主要落点：

- `map_nav_src/r2r/agent.py`
- `map_nav_src/r2r/parser.py`
- `map_nav_src/scripts/run_r2r.sh`

支持脚本层：

- `ANTI_LOOP_MODE=off|on`
- `ANTI_LOOP_EXTRA_ARGS="..."`

1.6 REVERIE: anti-loop 迁移

已经把 `R2R` 上的 next-hop anti-loop 逻辑迁到了 `REVERIE`：

- parser 开关已加
- `agent_obj.py` 的 rollout 路径已支持
- `run_reverie.sh` 已支持脚本层开关

主要落点：

- `map_nav_src/reverie/agent_obj.py`
- `map_nav_src/reverie/parser.py`
- `map_nav_src/scripts/run_reverie.sh`

迁移说明文档：

- `codex/reverie_decay_antiloop迁移说明.md`

2. 目前各条实验线的阶段性结论

2.1 R2R 上，dynamic memory 的主结论

整体看下来：

- `decay_only` 是当前最稳、最值得继续的主线
- `update_only / full` 可以做研究性尝试，但更容易引入副作用

一个很清晰的经验是：

- `decay_only` 更像“去掉陈旧 / 冗余 / 噪声 memory 的影响”
- `update/full` 更像“主动改 memory 内容”

在无训练适配条件下：

- 被动去噪通常更稳
- 主动改写通常更容易犯错

2.2 R2R 上，anti-loop 的主结论

这条线不是没用，而是更像：

- 用少量分数换更短路径、更高执行效率

结论可分成两种用途：

如果目标是最好 benchmark 分数

主推荐：

- `decay_only`

如果目标是更短路径、更快评测、更少即时回头

备选配置：

- `decay_only + immediate backtrack suppression`

当前比较清楚的结论是：

- `anti-loop` 不一定能把最终 `SR` 再往上推
- 但它能比较稳定地压缩：
  - `lengths`
  - `steps`
  - `cost time`

也就是说，它更像一个效率版 / 行为更整洁版，而不是最高分版。

这条线已经整理成中文总结：

- `codex/decay_antiloop_summary.md`

2.3 REVERIE 上，decay_only 与 decay_only + anti-loop

目前我们已经在 `REVERIE` 上跑到一组不错的：

- `final_decay_only_antiloop`

和之前的：

- 参数组1_decay_only

相比，结论不是“全面更好”，而是：

- `val_train_seen` 基本不变
- `val_seen` 小幅下降
- `val_unseen` 明确提升

也就是说：

> `REVERIE decay_only + anti-loop` 的价值更偏向提升 unseen 泛化，而不是提升 seen。

如果更看重泛化，这组是有意义的。

3. 当前推荐的主配置与备选配置

3.1 R2R 当前主配置

如果目的是主结果 / 主报告：

- `DYNAMIC_MEMORY_MODE=decay_only`
- 不加 anti-loop

这是当前分数导向下最稳的配置。

3.2 R2R 当前备选效率配置

如果目的是：

- 路径更短
- 步数更少
- 评测更快

可以用：

- `DYNAMIC_MEMORY_MODE=decay_only`
- `ANTI_LOOP_MODE=on`

其中更像工作点的是：

- `anti_loop_backtrack_penalty = 0.22` 或 `0.28`
- `anti_loop_revisit_penalty = 0.0`
- `anti_loop_min_step = 1`

但最终从 score 来看，主线仍然是 `decay_only`。

3.3 REVERIE 当前推荐理解

`REVERIE` 这边更适合把：

- `decay_only + anti-loop`

理解成一个“更偏 unseen 泛化”的配置。

如果后面要继续推进 `REVERIE`，建议优先盯：

- `val_unseen`
- `sr / spl / rgs / rgspl`

而不是只看 `val_seen`

4. 到目前为止，不建议再继续深挖的方向

当前不建议作为主线继续推进的方向有：

4.1 dual STOP

问题：

- 更容易导致“到了但不让停”
- `oracle_sr` 高，但 `sr / spl` 下降

4.2 instruction augmentation

问题：

- 明显扰动 instruction 分布
- 在无训练条件下风险太大

4.3 最小 instruction rerank

问题：

- 要么太弱，不改动作
- 要么能改动作，但不能带来更好结果

这些方向保留研究记录即可，不建议继续当主优化线。

5. 现阶段最值得继续的工作

如果继续做最小 test-time 改动，优先级建议是：

1. 继续把 `decay_only` 当主线
2. 把 `anti-loop` 当作效率 / 行为约束线，而不是主分数线
3. 在 `REVERIE` 上继续观察 `anti-loop` 对 `unseen` 的影响
4. 暂时不要再回到 instruction 线和 dual STOP 线

6. 相关文档索引

目前 `codex` 里和这批实验最相关的文档有：

- `codex/dynamic_memory_cfg.md`
- `codex/instruction_line_summary.md`
- `codex/decay_antiloop_summary.md`
- `codex/reverie_decay_antiloop迁移说明.md`

如果后面继续做：

- `R2R decay_only`
- `R2R decay_only + anti-loop`
- `REVERIE decay_only + anti-loop`

建议就在这几份总结的基础上往下迭代，不再把已经证伪的分支重新混进主线。